Module 4

LoRa Peer-to-Peer & LoRaWAN via ANTARES



IRIS Laboratory

LoRa Theory

LoRa: A Modulation Technique, Not a Network Protocol

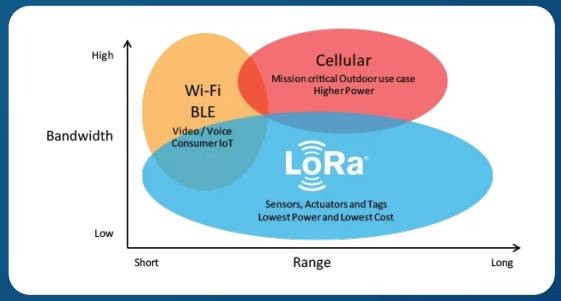
LoRa (**Long Range**) is a **spread spectrum radio modulation technique**, not a network protocol. Its key traits:

- **Long** range, **low** data rate, **low** power consumption.
- Operates in the **sub-GHz** band, whose regulations differ by region.

Example applications: smart agriculture · smart cities · industrial monitoring · environmental sensing

LoRa is **not** a network protocol. The network protocol commonly built on top of LoRa is **LoRaWAN**.

Where LoRa, CSS, and Its Parameters Fit



LoRa compared to Wi-Fi and BLE (Modules 1-2): long range, low data rate.

LoRa uses **Chirp Spread Spectrum (CSS)** — frequency sweeps up and down over time, so the signal stays recognizable below the **noise floor**.

Spreading Factor (SF) trade-off:

SF up ⟶ range up, sensitivity up, **airtime** up, data rate **down**.

This practicum uses SF12

Airtime & AS923-2 Regulation

Airtime is how long a single packet stays on air while being transmitted. It matters because it affects:

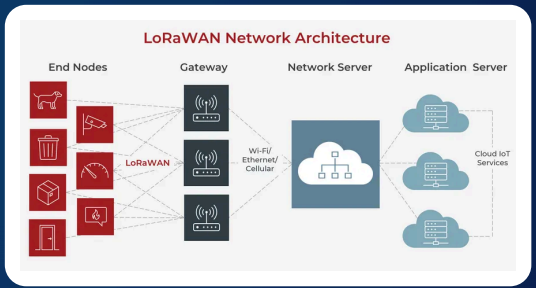
- Compliance with the **duty cycle**
- The chance of **collisions** between packets
- The device's **battery consumption**

Airtime will be measured directly in the LoRaWAN experiment.

Regional band plans: **EU863-870** (Europe), **US902-928** (Americas). Indonesia uses **AS923-2**, the **920-923 MHz** band → code `LORA_FREQUENCY 920E6`. The values `433E6`, `866E6`, `915E6` belong to other regions — do not use them.

LoRa vs LoRaWAN & Its Architecture

LoRa = the physical/modulation layer.
LoRaWAN = the network protocol built on top of it (join, security, authentication, forwarding to the application).



- **P2P**: transmitter and receiver must agree on frequency, bandwidth, SF, coding rate, and sync word.
- **LoRaWAN**: one packet can be heard by several gateways, then forwarded over IP.

Four components: **end device** · **gateway** · **network server** · **application server**

Device Classes & ABP Activation

Class determines when the RX window opens ⟶ affects latency and power.

- **A**: uplink-initiated, two RX windows, most power-efficient, mandatory for all devices. **(used in this practicum)**
- **B**: beacon + scheduled ping slots.
- **C**: RX almost always open, lowest latency, highest power draw.

ABP vs **OTAA**:

- **ABP**: session keys are hard-coded, no join process. **(used in this practicum)**
- **OTAA**: join-request/join-accept, per-session keys, more secure.

ABP keys: `DevAddr`, `NwkSKey`, `AppSKey`.

Keys are **confidential** — never write the real values into a journal, report, or public repository.

Implementation — The LoRa Hunt

Why There Are No Time Slots Any More

One **Beacon** board is hidden and transmits continuously. Six **Hunter** boards only **listen**, and report the signal strength they catch to the scoreboard over Wi-Fi.

Only **one radio** in the room ever transmits, so a packet collision is impossible. The elaborate per-group second-slot scheme is no longer needed — not because it was fixed, but because the topology means the problem never arises.

This is the module's central comparison: in **Part A** you build every network rule yourselves, while in **Part B** LoRaWAN has already supplied them.

Beacon — Radio Parameters

```
#define LORA_FREQUENCY          920E6
#define LORA_SPREADING_FACTOR   7
#define LORA_BANDWIDTH          250E3
#define LORA_SYNC_WORD          0xF3
```

Frequency, SF, Bandwidth, Sync Word

```
#define BEACON_TX_POWER 2

const unsigned long BEACON_PERIOD_MS = 200;
```

Transmit Power + Interval

The transmit power is deliberately **2 dBm**, not 20 dBm. At full power the whole room reads as equally strong and the beacon is impossible to find — turning the power **down** is what makes the game work.

Beacon — Radio setup()

```
pinMode(LORA_EN, OUTPUT);
digitalWrite(LORA_EN, HIGH);  // Switch the
radio enable circuit on
delay(100);

SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI,
LORA_NSS);
LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);
```

Enable the Radio + SPI

```
LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);
LoRa.setSignalBandwidth(LORA_BANDWIDTH);
LoRa.setSyncWord(LORA_SYNC_WORD);
LoRa.setTxPower(BEACON_TX_POWER);
```

Modem Parameters + Power

All four radio values must be **identical** on every Hunter. If even one differs, the packets do not error — they are simply **silent**.

Beacon — `loop()`: Transmitting

```
if (millis() - lastChirp < BEACON_PERIOD_MS) {
  return;
}

lastChirp = millis();
seq++;

String sentAt = wallClockTimestamp();
String payload = String(seq) + " " + sentAt;
```

200 ms Interval + Packet Number

```
LoRa.beginPacket();
LoRa.print(payload);
LoRa.endPacket();
```

Build & Transmit

`beginPacket()` prepares the buffer and `print()` fills it, but it is **`endPacket()`** that actually puts the packet on the air. Every packet carries a **sequence number** and its **send time**.

Hunter — Library & Radio Pins

```
#include <WiFi.h>
#include <SPI.h>
#include <LoRa.h>
#include <WebSocketsClient.h>
#include <time.h>
#include <sys/time.h>
```

Library Imports

```
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32
#define SPI_MISO  19
#define SPI_MOSI  23
#define SPI_SCK   18
```

RFM95W Pins

GPIO **15, 0, 27, 32, 18, 19** and **23** are already taken by the RFM95W radio module. A Hunter needs no voltage divider because nothing is wired to it.

Hunter — Smoothing the RSSI

```
int median() {
  int s[MEDIAN_N];
  memcpy(s, window, windowCount * sizeof(int));

  for (int i = 1; i < windowCount; i++) {
    int key = s[i], j = i - 1;
    while (j >= 0 && s[j] > key) { s[j + 1] =
s[j]; j--; }
```

Median of 5 Readings

```
int m = median();
if (!haveSmoothed) {
  smoothed = m;
  haveSmoothed = true;
} else {
  smoothed = EWMA_ALPHA * m + (1 - EWMA_ALPHA) *
smoothed;
}
```

EWMA on Top of It

Indoors, RSSI jumps by as much as **10 dB** from wall reflections. Without both filters the distance bar is nothing but noise.

Hunter — `loop()`: Catch & Report

```
String beaconLine = "";
while (LoRa.available()) {
  beaconLine += (char)LoRa.read();
}

String receivedAt = wallClockTimestamp();

int rssi = LoRa.packetRssi();
float snr = LoRa.packetSnr();
```

Read Packet + RSSI

```
String seq    = beaconLine.substring(0, space);
String sentAt = beaconLine.substring(space + 1);

webSocket.sendTXT(payload);
```

Report over Wi-Fi

A Hunter's radio **never transmits**. The only thing leaving this board is a Wi-Fi report to the scoreboard — which is exactly why all six groups can hunt at the same time without interfering.

Scoreboard — The Application Layer

One RSSI reading crosses five layers before it becomes a number on screen:

Physical the 920 MHz wave, SF7, 250 kHz — RSSI and SNR
Data link LoRa packet numbering, lost packets, sync word
Network an IP address from the Orange Pi's DHCP, one hop to its gateway
Transport one WebSocket connection over TCP
Application the ranking, the heat, and the winner

The scoreboard at `http://192.168.4.1:8080` shows all five layers with their live values, per group.

</> Babai </>
#IRIS